

Table of content

1. Introduction.....	1
2. Tools and Technology.....	2
3. Dataset Description.....	6
4. Section - 1 (Data Understanding).....	7
5. Section - 2 (Data Preparation).....	10
5a - Loading and 50 records of the data.....	10
5b - Filtration of a DataFrame.....	11
5c - Checking and Removal of Missing Values.....	12
5d - Checking Duplicate Records in existing data.....	13
5e - Display of the Number of Columns and Names.....	14
5f - Renaming Label Values.....	15
6. Section - 3 (Data Preparation).....	16
6a - Summary statistics of sum, mean, standard deviation, skewness, and kurtosis.....	17
6b - Correlation of all numeric variables and display top 5 correlated features.....	19
7. Section - 4 (Data Exploration).....	22
7a - Label Frequency Analysis.....	23
7b - Comparison of Average Flow Duration and Packet Length Mean.....	25
7c - Boxplot of 'Fwd Packet Length Mean' grouped by the Label column.....	27
7d - Hypothesis test to determine difference in the mean Flow Duration.....	28
7e - Dashboard.....	30
8. Summary.....	31
9. Conclusion.....	32

Table of figure

Figure no. 1 - Python.....	2
Figure no. 2 - pandas.....	2
Figure no. 3 - matplotlib.....	3
Figure no. 4 - seaborn.....	3
Figure no. 5 - scipy.....	4
Figure no. 6 - numpy.....	4
Figure no. 7 - jupyter notebook.....	5
Figure no. 8 - Read csv.....	10
Figure no. 9 - Filter df.....	11
Figure no. 10 - Missing values.....	12
Figure no. 11 - Duplicates.....	13
Figure no. 12 - New columns.....	14
Figure no. 13 - Label values.....	15
Figure no. 14 - statistic summary.....	17
Figure no. 15 - statistic summary output.....	18
Figure no. 16 - correlation matrix.....	19
Figure no. 17 - correlation values.....	20
Figure no. 18 - top 5 correlated features.....	21
Figure no. 19 - value counts.....	23
Figure no. 20 - bar chart of value counts.....	24
Figure no. 21 - Average flow and packet mean.....	25
Figure no. 22 - bar chart.....	26
Figure no. 23 - boxplot of fwd packet length.....	27
Figure no. 24 - hypothesis testing.....	28
Figure no. 25 - hypothesis test result.....	29
Figure no. 26 - Dashboard.....	30

1. Introduction

Network security is now one of the most significant issues in the digital world where organisations, governments, and even individuals are in the fast-changing digital world. As the use of the internet and connected devices grows exponentially the amount of network traffic has increased significantly and it has become even more challenging to monitor and detect malicious activity manually. The Intrusion Detection Systems (IDS) have become an essential part of the sphere of cybersecurity that allows detecting suspicious and malicious network behaviour automatically.

The dataset comprises the records of network flows obtained on the actual network traffic and each record describes a single connection between a source machine and a destination machine. The network flows are marked as BENIGN, as it is normal traffic, or as an attack as it is malicious traffic and it may be an infiltration attempt. The main aim of the analysis is to better understand the network flow properties, discover significant relationships among attributes, and pre-process the data to lead to the next step that is to additional machine learning model training to detect intrusion.

The analysis is done using Python programming language, which is said to be one of the most influential and universal data analysis and machine learning tools. Various Python libraries were also used during this project, such as Pandas to manipulate and clean data, Matplotlib and Seaborn to visually depict data and SciPy to test statistical hypotheses. All the code was developed and run in a Jupyter notebook environment, which had an interactive and structured development and documentation environment.

This report will be divided into four major sections. The former section is data understanding and includes a description of the dataset and its main characteristics. The second section is concerned with data preparation that covers loading, filtering, cleaning and transforming the data. The third part is the data analysis, which presents the summary statistics and correlation analysis. The fourth and last part is the data exploration, which entails a visual analysis in the form of charts and a formal hypothesis test.

2. Tools and Technology

Python

This entire analysis was done using the Python 3.11 programming language. Python is considered to be one of the most popular, powerful, and general purpose programming languages in the world particularly regarding data science, machine learning, and artificial intelligence. The best project to use in data analysis is because it is simple, easy to read and has a huge library that supports the project. The efficiency of Python in dealing with large volumes of data, as well as the capability to deal with a large variety of open-source libraries, was the reason why this language was chosen as the most appropriate one to work on this coursework. Moreover, Python is popular in the industry and academia and as such, it can be a useful skill acquired by any data scientist-software developer.



Figure no. 1 - Python

Pandas

Pandas is a strong open-source data analysis and manipulation python library. It offers data structures like DataFrames and Series with which it is a breeze to work with structured data. During this project, Pandas was applied to perform a large number of tasks such as loading the dataset in a CSV file into a DataFrame, selecting and filtering particular columns, checking and eliminating the missing values, finding and counting the duplicate records, renaming label values, and other data transformation functions. The process of data preparation would have required much more time and would have been much more complicated without Pandas.



Figure no. 2 - pandas

Matplotlib

One of the best known and most popular libraries of Python to produce data visualisation of various types (static, animated, and interactive) is Matplotlib. It offers the entire package of the tools to generate any form of chart and graph. All of the visualisations created in this project were done in Matplotlib such as the bar chart of the frequency of each label category, pie charts of the average Flow Duration and of the Packet Length Mean by label and the boxplot of the distribution of Fwd Packet Length Mean by label. Such visualisations were important to aid in the presentation of the analysis outcomes in a simple and visually attractive way.

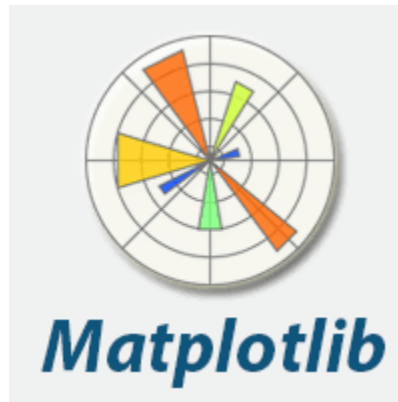


Figure no. 3 - matplotlib

Seaborns

Seaborn is a Python data visualisation library, which is an extension of Matplotlib that offers a more user-friendly interface to more visually appealing and informative statistical diagrams. In this project, the use of Seaborn helped to improve the visual quality and aesthetics of charts and plots created in the course of data exploration. The visualisations were also made more professional and easier to interpret due to its built-in themes and colour palettes.



Figure no. 4 - seaborn

SciPy

SciPy is an open-source scientific computing library of Python which offers extensive tools of mathematics, science and engineering. In this project, it was applied specifically in the hypothesis testing part where it was used to conduct the independent samples t-test. The test statistic that was conducted was a t-test to ascertain whether a statistical significant difference existed between the means of the Flow Duration in case of normal Traffic and Attack classes. The t-statistic and p-value required to interpret the results could be easily created using the SciPy stats module and the t-test was easily run on the module.



Figure no. 5 - scipy

Numpy

NumPy is a Python scientific computing core library that has support for large multi-dimensional arrays and matrices, and a set of mathematical functions. NumPy was also not applied directly in the current project but was used in the background as both Pandas and SciPy are built on it, allowing efficient and quick numerical computing throughout the analysis.



Figure no. 6 - numpy

Jupyter Notebook

The main development and documentation environment of this whole project was Jupyter Notebook. Jupyter notebook is a free visual interactive computing platform built upon the concept of an open-source web-based application which enables users to establish and distribute files containing live code, visualisations, narrative text, and mathematical equations within one document. It gave an easy and interactive and structured platform whereby the code, outputs, charts and written explanations could be managed effortlessly to amalgamate to give the analysis structured, readable and easy to follow. Jupyter Notebook is popular with data scientists and assumed to be one of the most suitable tools to perform exploratory data analysis and academic reporting.



Figure no. 7 - jupyter notebook

3. Dataset Description

The data used in the analysis is the CICIDS (Canadian Institute For Cybersecurity Intrusion Detection System) Network Traffic Dataset. The dataset is reputable and utilized widely in the research of cybersecurity and it is considered to be one of the most complete and realistic network traffic data to be utilized in intrusion detection research. It was developed by Canadian Institute for Cybersecurity at University of New Brunswick and has been used by researchers and data miners around the globe in developing and testing machine learning based intrusion detection systems.

The dataset has 288,602 records and 79 features with each row representing a single network flow (communication session between a source and a destination machine) on the network. Any network flow is determined by a huge number of statistical features that characterize the behaviour and nature of the connection, such as the length of the flow, the amount of packets sent and received, their size, the rate of data transfer, etc.

The dependent variable of the data set is the Label column that denotes all network flows as BENIGN, which denotes normal and legitimate traffic on the network, or an attack that includes malicious and suspicious network traffic such as attempts to infiltrate the network. To carry out the analysis, the name of the label was changed to reflect the values of the label as Normal Traffic and Attack respectively to make them easier to read and understand.

The features contained in the dataset are a huge amount that can be grouped as the flow-based features (Flow Duration, Flow Bytes per second), the packet-based features (Total Forward Packets, Packet Length Mean), and the time-based features (Active Mean, Idle Mean). A combination of these features provides a detailed and rich picture of the network behaviour which may be readily used to sketch the line between the normal and malicious traffic.

4. Section - 1 (Data Understanding)

To gain the most insight, the first and most crucial step in any data analysis or data mining project is understanding the dataset. Before data preparation, analysis, or even modeling, one should know the dataset, its structure, and its features, as well as the value each variable holds. This part will give the reader the understanding needed for the dataset concerning this analysis and the most relevant columns to network traffic and intrusion detection.

The analysis for this study employed the CICIDS network traffic dataset which has 288,602 instances with 79 features. Each instance in the dataset describes one network flow or one communication session between one source and one destination machine across the network. The dataset also has both attack and normal traffic and therefore can be used in binary classification tasks to determine whether the network activity is legitimate or malicious.

For this analysis, the dataset was reduced to 16 attributes deemed most relevant and informative and worthy of consideration. A deliberate effort was made in selecting the attributes to ensure optimal representation of the dataset in the most comprehensive and balanced manner, and to cover all areas of the flow, packet, and time characteristics of the network traffic data. Discussion of the individual attributes and their relevance to network traffic data analysis will follow in the next section.

Destination Port

Destination Port The port number in the destination machine that the network flow has been sent to. Each service and application in the machine is identified using port numbers. Attackers will often target specific port numbers, e.g. port 80 (which carries HTTP traffic) and port 443 (which carries HTTPS traffic), and this capability may be used to detect bad behavior.

Flow Duration

Flow Duration is the sum of the duration which the network flow takes in microseconds. The length of a network flow can also be a valuable time-based indicator of suspicious behavior because attack traffic can either be unusually short or unusually longer than regular traffic.

Total Fwd Packets

Total Fwd Packets is the sum total of the number of packets transmitted in the forward direction, or in the direction of the source machine to the destination machine. An increase in forward packets can either mean massive transfer of data or a possible data flooding attack.

Total Backward Packets

Total Backward Packets, the total number of packets that are sent on the backward route, i.e. in the direction of the sending machine to the destination machine. The rate of forward-forward packets to backward-forward packets may give helpful data on the type of conversation.

Total Length of Fwd Packets

Total Length of Fwd Packets is used to describe the sum of all the packet sizes that are forwarded. This feature gives the information of the amount of data being sent between the origin and the destination.

Total Length of Bwd Packets

Total Length of Bwd Packets - The sum of all the sizes in bytes of packets sent in the reverse direction. This along with the total length of forward packets aid in characterising the overall size and direction of data movement in the network stream.

Fwd Packet Length Mean

Fwd Packet Length Mean is the mean length of packets sent towards the forward direction. The length of packets can also be of helpful data in regard to the nature of application or protocol being utilized, or even indicative signs of malicious intent.

Bwd Packet Length Mean

Bwd Packet Length Mean - It is the average length of the packets that are transmitted in the opposite direction. Just like the forward packet length means, this option could be used to describe the nature of the network communication.

Flow Bytes

Flow Bytes per Second This rate is the number of bytes being transferred per second in the flow of the network. A high flow bytes per second value is unusual and could indicate an attempt to exfiltrate data or could indicate a denial of service attack.

Flow Packets

Flow Packets per Second is a measure of the quantity of packets that are sent across the flow of the network each second. A high flow packet per second might be an indication of a denial of service attack or flooding.

Packet Length Mean

Packet Length Mean The means of the packet lengths of the packet flow in the network, regardless of the direction. It is this attribute that approximates the size characteristics of packet flow.

Packet Length Std

Packet Length Std The standard deviation of the network flow packet sizes. High standard deviation indicates there is a big gap between the sizes of packets, and this gap may be explained by the uneven or suspicious nature of traffic.

Average Packet Size

The mean size of all the packets of the network flow is known as the Average Packet Size. The feature is similar to Packet Length Mean and provides additional information regarding the overall characteristics of the packet size of the flow.

Active Mean

Active Mean The average number of seconds that the network flow was active and then became idle. This property is applied to record the timing properties of the flow and this can be used to identify other forms of network activity.

Idle Mean

Idle Mean This is the mean amount of time that the flow on the network remained idle, and then it was active again. This characteristic together with Active Mean provide effective data regarding the time behaviour of the network flow.

Label

The dependent variable of the dataset is the label. It classifies all network flows as its Normal Traffic that is legitimate and benign network activity or Attack which is malicious and suspicious network activity. It is a dependent variable and against which the machine learning model will be trained to make predictions.

5. Section - 2 (Data Preparation)

Data preparation is one of the most critical and time-consuming data mining or data analysis project processes. This entails the transformation of dirty and raw data into a clean and structured and conducive form that is easy to conduct an analysis, visualisation, and train machine learning models. Unprepared data may lead to the unstable, inaccurate and misleading results of any analysis. The section details all the operations of the data preparation, which was carried out on the network traffic data, e.g., the loading of the data, the deletion of the undesired columns, the manipulation of the missing value, the identification of the duplicate lines, the demonstration of the column names, and renaming of labels values.

5a - Loading and 50 records of the data.

Pandas DataFrame was used to read in network traffic, read csv-function was implemented and this was the first step in the data preparation process. DataFrame The DataFrame is a two-dimensional table of data provided by the Pandas library that implies that the data may be stored and manipulated in rows and columns like in a spreadsheet. Once the dataset has been loaded successfully, the tail(50) command was invoked, such that 50 records of the data would be displayed. The tail command will result in the records of the last rows of the DataFrame and it is not hard to verify that one has properly loaded the data and a rough notion of how and what the data is in the very end of the file.

```
[1]: import pandas as pd
[2]: df = pd.read_csv('/kaggle/input/datasets/akisavujel/network-traffic/ids.csv')
[3]: df.tail(50).style\
      .background_gradient(cmap='Purples')\
      .highlight_null(color='red')\
      .set_caption('Filtered Dataset')\
      .set_table_styles([
          {'selector': 'caption',
           'props': [
               ('font-size', '14px'),
               ('font-weight', 'bold'),
               ('color', '#2d5a4f')]
          }])
```

	Unnamed: 0	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Fwd Packet Length Std	Bwd Packet Length Max	Bwd Packet Length Min	Bwd Packet Length Mean	F Leng
488065	488065	53	70591	2	2	66	194	33	33	33.000000	0.000000	97	97	97.000000	0.0i
488066	488066	443	5572560	8	6	386	5205	205	0	48.250000	75.952523	2021	0	867.500000	883.0
488067	488067	3013	49	2	0	4	0	2	2	2.000000	0.000000	0	0	0.000000	0.0i
488068	488068	53	122130	2	2	80	272	40	40	40.000000	0.000000	136	136	136.000000	0.0i
488069	488069	53	98106274	2	2	114	211	58	56	57.000000	1.414214	139	72	105.500000	47.3
488070	488070	53	506	1	1	53	113	53	53	53.000000	0.000000	113	113	113.000000	0.0i
488071	488071	53	3957631	2	2	120	260	79	41	60.000000	26.870058	161	99	130.000000	43.8
488072	488072	9102	2	2	0	4	0	2	2	2.000000	0.000000	0	0	0.000000	0.0i
488073	488073	2135	111	2	2	4	12	2	2	2.000000	0.000000	6	6	6.000000	0.0i
488074	488074	1048	49	2	0	4	0	2	2	2.000000	0.000000	0	0	0.000000	0.0i
488075	488075	54240	55	1	1	6	6	6	6	6.000000	0.000000	6	6	6.000000	0.0i
488076	488076	1049	73	2	2	4	12	2	2	2.000000	0.000000	6	6	6.000000	0.0i

Figure no. 8 - Read csv

5b - Filtration of a DataFrame.

The original information was huge in terms of size and the majority of the columns were not needed per the purpose of this analysis. It was as a result of this that the new filtered DataFrame had only 16 key columns as this made the analysis only deal with the most valuable and interesting characteristics. These 16 columns have been adopted to be inclusive of all the 3 major groups of features that include the flow based features, the packet based features and the time based features. When working with Pandas, there was column indexing to generate the new filtered DataFrame. The resulting DataFrame with the result of the filtering process had 16 columns and 488,115 rows which confirmed the fact that the filtering process was achieved.

```
columns = ['Destination Port', 'Flow Duration', 'Total Fwd Packets',
           'Total Backward Packets', 'Total Length of Fwd Packets',
           'Total Length of Bwd Packets', 'Fwd Packet Length Mean',
           'Bwd Packet Length Mean', 'Flow Bytes/s', 'Flow Packets/s',
           'Packet Length Mean', 'Packet Length Std', 'Average Packet Size',
           'Active Mean', 'Idle Mean', 'Label']
```

```
df_filtered = df[columns]
```

```
print("=" * 60)
print("    ✅ FILTERED DATAFRAME CREATED SUCCESSFULLY!")
print("=" * 60)
print("    Total Rows      :", df_filtered.shape[0])
print("    Total Columns   :", df_filtered.shape[1])
print("=" * 60)
```

```
=====
✅ FILTERED DATAFRAME CREATED SUCCESSFULLY!
=====
```

```
Total Rows      : 488115
Total Columns   : 16
=====
```

Figure no. 9 - Filter df

5c - Checking and Removal of Missing Values.

The data quality problems like the missing values can have an enormous impact on the quality and validity of the data analysis. In order to identify the existence of any missed values in the filtered DataFrame, `isnull()` function was used to identify the presence of any column where the values of the column were either truly null or NaN. The results provided showed that there were 36 blank values in the column of Flow Bytes/s, and none in the other 15 columns. It was the resultant sum of all the missing values of the complete filtered DataFrame, which was 36. The 36 rows also had gaps that were filled in the dataset using the `dropna()` function that eliminated the 488,115 rows to leave 488,079 rows. The additional analysis will be done using complete and valid data since the missing data will be eliminated.

```
print("=" * 50)
print(" 🔍 TOTAL MISSING VALUES")
print("=" * 50)
print(df_filtered.isnull().sum().sum())
```

```
=====
 🔍 TOTAL MISSING VALUES
=====
36
```

```
df_filtered = df_filtered.dropna()

print("=" * 50)
print(" ✅ SHAPE AFTER REMOVING MISSING VALUES")
print("=" * 50)
print(df_filtered.shape)
```

```
=====
 ✅ SHAPE AFTER REMOVING MISSING VALUES
=====
(488079, 16)
```

Figure no. 10 - Missing values

5d - Checking Duplicate Records in existing data.

This is among the formal checks that should be fulfilled by all the models. The other common data quality issue is duplicate records which can negatively affect the final result of the data analysis process and machine learning model training. Duplicate observation: This occurs when a specific observation is keyed into the database more than once and this can provide biased and incorrect information. To identify the duplicates in the filtered DataFrame, a duplicated function was used to test them and see whether all the rows in DataFrame were the ideal copies of its predecessor. Each row obtained a False or True value where the True value represented a row was a duplicate. The number of duplicate records was then added up using the sum function. As it happened the filtered DataFrame had 139,389 duplicate records and it is rather significant. The fact that this is simply staggering demonstrates the fact that one needs to check the data quality thoroughly before proceeding further with any other analysis since it is very likely that the occurrence of duplicates in the data set is likely to provide biased results.

```
print("=" * 50)
print("  🔄 DUPLICATE RECORDS")
print("=" * 50)
print(df_filtered.duplicated())

=====
  🔄 DUPLICATE RECORDS
=====
0      False
1      False
2      False
3      False
4      False
...
488110   True
488111   True
488112  False
488113  False
488114  False
Length: 488079, dtype: bool

print("=" * 50)
print("  🔄 TOTAL DUPLICATE RECORDS")
print("=" * 50)
print(df_filtered.duplicated().sum())

=====
  🔄 TOTAL DUPLICATE RECORDS
=====
139389
```

Figure no. 11 - Duplicates

5e - Display of the Number of Columns and Names.

The filtering and cleaning of the database was then presented with the column count and the column names to make sure that the outlay is correct. The latter attribute `shape[1]` was then resorted to to provide the number of columns in the filtered DataFrame that revealed the presence of 16 columns. This was succeeded by a loop to indicate the names of the columns and invoking the `enumerate()` function that in turn invoked another function to add the names of the columns and the numbers assigned to the columns. The filtered DataFrame contains 16 columns; these are the columns as follows Destination Port, Flow Duration, Total Fwd Packets, Total Backward Packets, Flow Length Mean, Flow Length Std, Packet Length Mean, Packet Length Std, Average Packet Size, Active Mean, Idle Mean and Label. This was done by making sure that the right number of columns were in the right place as well as identifying the right ones.

```
print("=" * 50)
print(" 📊 TOTAL NUMBER OF COLUMNS")
print("=" * 50)
print(df_filtered.shape[1])
```

```
=====
📊 TOTAL NUMBER OF COLUMNS
=====
16
```

```
print("=" * 50)
print(" 📋 COLUMN NAMES")
print("=" * 50)
for i, col in enumerate(df_filtered.columns, 1):
    print(f"{i}. {col}")
```

```
=====
📋 COLUMN NAMES
=====
1. Destination Port
2. Flow Duration
3. Total Fwd Packets
4. Total Backward Packets
5. Total Length of Fwd Packets
6. Total Length of Bwd Packets
7. Fwd Packet Length Mean
8. Bwd Packet Length Mean
9. Flow Bytes/s
10. Flow Packets/s
11. Packet Length Mean
12. Packet Length Std
13. Average Packet Size
14. Active Mean
15. Idle Mean
16. Label
```

Figure no. 12 - New columns

5f - Renaming Label Values

The last undertaking of the data preparation was the transformation of the values of the Label column into more descriptive and intuitive and understandable names. Prior to the renaming, there were two additional values in the column of Label that comprises BENIG and Infiltration. Such values have been re-nominated with replacement normal Traffic and Attack respectively through replace () panda function. The replace method enables an opportunity to substitute certain values in one of the columns with other values without changing the other values in the DataFrame. The unique was then invoked to ensure that the renaming process was done properly and it was therefore pointed out that the Label column now contained only the two values of the values being the Normal Traffic and Attack. Value count was then applied to illustrate the number of records in each category of labels to obtain a bare image of the distribution in the two classes in the data set. The labels are made so much easier to read and understand once the analysis is complete and with all the visualisations with the re-naming.

```
df_filtered['Label'] = df_filtered['Label'].replace({
    'BENIGN': 'Normal Traffic',
    'Infiltration': 'Attack'
})
```

```
print("=" * 50)
print("    LABEL VALUES AFTER RENAMING")
print("=" * 50)
print(df_filtered['Label'].unique())
```

```
=====
    LABEL VALUES AFTER RENAMING
=====
['Normal Traffic' 'Attack']
```

```
print("=" * 50)
print("    LABEL VALUE COUNTS")
print("=" * 50)
print(df_filtered['Label'].value_counts())
```

```
=====
    LABEL VALUE COUNTS
=====
Label
Normal Traffic    288548
Attack            199531
Name: count, dtype: int64
```

Figure no. 13 - Label values

6. Section - 3 (Data Preparation)

In deriving meaningful information in the datasets, data analysis is important. The use of summary statistics is one significant element of data analysis because it is a convenient summary of the characteristics of numeric variables. Sum, mean, standard deviation, skewness, and kurtosis are some of the common measures that can be used when analyzing two selected numeric variables. The sum calculates the total values of all values, and the mean is an average that assists in comprehending the central tendency of the values used. The standard deviation used values are used to measure the variability of the values about the mean, or the extent to which the values vary about the mean.

Besides those, skewness and kurtosis indicate the form of the data distribution. Skew is used to tell whether the data is skewed or inclined on either side. A positive skewness indicates that the data is being stretched to the right whereas skewness is negative which implies a leftwards stretch. Instead, kurtosis is used to quantify the tailedness of the distribution, i.e., whether the data consists of extreme values or outliers. A combination of these statistics aids in appreciating the distribution as well as the behavior of the variables.

Correlation analysis is another vital aspect of data analysis that is used to determine the relationships among numeric variables. The values of correlation are between -1 and +1 with a value near +1 signifying strong positive relationship, a value near -1 signifying strong negative relationship and a value near zero signifying no or less relationship. Through computing the correlation among all the numeric variables, the analysts are able to detect patterns and dependencies in the data.

It is also useful to display the top five correlated features that are displayed, as it shows the strongest relationships. This assists in feature selection, predictive modeling and decision-making. In general, the summary statistics and correlation analysis are essential tools that allow better interpreting data and make the analysis effective.

6a - Summary statistics of sum, mean, standard deviation, skewness, and kurtosis

During the data analysis stage, Python code was written with the help of the pandas library in Jupyter notebook to create an extensive summary statistic of two well-chosen numeric variables of the Network Traffic data. Flow Duration and Packet Length Mean were selected. Flow Duration was chosen, as it is the overall time duration of each individual network flow and offers crucial information about connection behaviour, speed of sessions and possible anomalies like longer than usual traffic. Packet Length Mean was selected because it shows the mean size of packets sent during the communication process, which is quite useful when analyzing the pattern of data transmission and detecting anomalies in network processes.

The program calculated the five main statistical measures of the sum, mean, standard deviation, skew and kurtosis. Such statistics contributed to the uncovering of central tendency, dispersion and form of data distribution. The findings showed that the two variables have a high positive skewness meaning they are highly skewed to the right with a long tail of extreme values. Also, the extremely large values of kurtosis indicated the existence of sharp peaks and heavy tails, which validated the existence of many outliers. The high standard deviation also led to the high variability in the data set. On the whole, it was found that the analysis performed by this summary statistics allowed gaining a solid background knowledge about the nature of the data to proceed to further correlations and training a machine learning model.

```
cols = ['Flow Duration', 'Packet Length Mean']
```

```
sum_result = df_filtered[cols].sum()
mean_result = df_filtered[cols].mean()
std_result = df_filtered[cols].std()
skew_result = df_filtered[cols].skew()
kurt_result = df_filtered[cols].kurt()
```

```
print("=" * 50)
print("  +  SUM")
print("=" * 50)
print(sum_result)

print("\n")

print("=" * 50)
print("  📊  MEAN")
print("=" * 50)
print(mean_result)

print("\n")

print("=" * 50)
print("  📉  STANDARD DEVIATION")
print("=" * 50)
print(std_result)

print("\n")

print("=" * 50)
print("  📈  SKEWNESS")
print("=" * 50)
print(skew_result)

print("\n")

print("=" * 50)
print("  📉  KURTOSIS")
print("=" * 50)
print(kurt_result)
```

Figure no. 14 - statistic summary

```

=====
+ SUM
=====
Flow Duration          4.364853e+12
Packet Length Mean    4.175763e+07
dtype: float64

=====
MEAN
=====
Flow Duration          8.942923e+06
Packet Length Mean    8.555506e+01
dtype: float64

=====
STANDARD DEVIATION
=====
Flow Duration          2.748182e+07
Packet Length Mean    1.738725e+02
dtype: float64

=====
SKEWNESS
=====
Flow Duration          3.250874
Packet Length Mean    4.935766
dtype: float64

=====
KURTOSIS
=====
Flow Duration          9.246791
Packet Length Mean    34.610512
dtype: float64

```

Figure no. 15 - statistic summary output

6b - Correlation of all numeric variables and display top 5 correlated features

Correlation analysis is used to determine the strength and direction of the relationships among variables in data. The program guarantees that analysis is meaningful and accurate, since correlation measures demand quantitative data because it is only numeric.

The steps start with the process of choosing all the numeric columns of the dataset. This is necessary since data sets have categorical or text data which cannot be utilized in correlation analysis. After isolating the numeric variables, the program calculates a Correlation matrix. This matrix gives the pairwise correlation values of each combination of variables which are usually between -1 and +1. A high value near +1 would depict that there is a high positive relationship implying that the higher the one variable, the higher the other one. On the other hand, a value close to -1 is a strong negative relationship, i.e. one variable is declining with an increase in the other. The values near zero indicate low or no relationship.

Once the correlation matrix has been created, the program will then run the results it receives to establish the most influential relationships. As there are redundant pairs and self-correlations (where a variable is fully correlated with itself) they are filtered out. The rest of the correlation values are then ranked in terms of their absolute value in order to point out the strongest associations, irrespective of direction.

Lastly, the most correlated pair of features are shown by the program. This is especially applicable in practice like feature selection, predictive modeling and interpretation of data. Strong correlations can either suggest helpful predictors or can help see a pattern in the data that was not immediately apparent. On the whole, such a Python code eases the complicated links in data and offers easy-to-understand insights, which is a valuable element of the exploratory data analysis model.

```
# The column label contains text variable which doesnot work with corr
correlation_matrix = df_filtered.drop(columns=['Label']).corr()
```

```
print("=" * 50)
print("  🔗 CORRELATION MATRIX")
print("=" * 50)
correlation_matrix.style.background_gradient(cmap='Greens')
```

Figure no. 16 - correlation matrix

Some positive correlations are evidently strong as indicated by the matrix. To use some specific examples, Total Fwd Packets and Total Backward Packets exhibit a high correlation (0.9498), and flow with high forward packets is likely to have a large number of backward packets also. On the same note, Total Backward Packets and Total Length of Bwd Packets (0.9615) have a high correlation, which is natural as with a high number of packets, the total length tends to increase mostly.

The other observation that is worth noting is the close association among the features related to the size of packet sizes. Packet Size Mean and Packet Size Average are calculated with very close correlation (0.9951) and indicate that they represent almost the same information. Similarly, Packet Length Std has a high correlation with Packet Length Mean (0.8878), which means that there are regular variations.

Also, there is a moderate to strong correlation between Flow Duration and Idle Mean (0.6531), making it appear that longer flows are usually characterized by longer idle times. In general, this matrix can be used to detect the redundant features and regarding significance of relationships, which will be handy in feature selection and enhancing the performance of models in data analysis activities.

CORRELATION MATRIX															
	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean
Destination Port	1.000000	-0.145250	-0.025711	-0.024028	0.004815	-0.015581	0.005199	-0.217264	0.063783	0.214177	-0.118523	-0.156795	-0.127719	-0.051505	-0.109348
Flow Duration	-0.145250	1.000000	0.124344	0.100836	0.032044	0.063465	0.114950	0.351037	-0.018884	-0.113359	0.324827	0.465984	0.303080	0.204673	0.653138
Total Fwd Packets	-0.025711	0.124344	1.000000	0.949848	0.333574	0.920007	0.035838	0.210119	-0.002078	-0.020480	0.200438	0.174919	0.190185	0.137132	0.074589
Total Backward Packets	-0.024028	0.100836	0.949848	1.000000	0.146728	0.961480	0.015679	0.200844	-0.002868	-0.020564	0.186629	0.156695	0.177258	0.101969	0.066117
Total Length of Fwd Packets	0.004815	0.032044	0.333574	0.146728	1.000000	0.013944	0.127556	0.016333	0.001838	-0.006509	0.080155	0.054492	0.077949	0.058811	0.013443
Total Length of Bwd Packets	-0.015581	0.063465	0.920007	0.961480	0.013944	1.000000	-0.001047	0.194512	-0.001385	-0.012061	0.176026	0.138066	0.167419	0.078208	0.048474
Fwd Packet Length Mean	0.005199	0.114950	0.035838	0.015679	0.127556	-0.001047	1.000000	0.123866	0.154969	-0.078665	0.612858	0.462664	0.628276	0.068762	0.074170
Bwd Packet Length Mean	-0.217264	0.351037	0.210119	0.200844	0.016333	0.194512	0.123866	1.000000	-0.024280	-0.166846	0.824952	0.876817	0.809420	0.187768	0.236769
Flow Bytes/s	0.063783	-0.018884	-0.002078	-0.002868	0.001838	-0.001385	0.154969	-0.024280	1.000000	0.200319	0.127875	0.069086	0.188463	-0.006707	-0.014174
Flow Packets/s	0.214177	-0.113359	-0.020480	-0.020564	-0.006509	-0.012061	-0.078665	-0.166846	0.200319	1.000000	-0.134655	-0.145674	-0.135588	-0.040319	-0.085080
Packet Length Mean	-0.118523	0.324827	0.200438	0.186629	0.080155	0.176026	0.612858	0.824952	0.127875	-0.134655	1.000000	0.887792	0.995068	0.182159	0.215195
Packet Length Std	-0.156795	0.465984	0.174919	0.156695	0.054492	0.138066	0.462664	0.876817	0.069086	-0.145674	0.887792	1.000000	0.875630	0.192092	0.270692
Average Packet Size	-0.127719	0.303080	0.190185	0.177258	0.077949	0.167419	0.628276	0.809420	0.188463	-0.135588	0.995068	0.875630	1.000000	0.171748	0.204645
Active Mean	-0.051505	0.204673	0.137132	0.101969	0.068811	0.078208	0.068762	0.187768	-0.006707	-0.040319	0.182159	0.192092	0.171748	1.000000	0.219721
Idle Mean	-0.109348	0.653138	0.074589	0.066117	0.013443	0.048474	0.074170	0.236769	-0.014174	-0.085080	0.215195	0.270692	0.204645	0.219721	1.000000

Figure no. 17 - correlation values

The output gives the five most strongly correlated pairs of features in the dataset indicating important relationships between network traffic variables. Packet Length Mean and Average Packet Size have the strongest correlation (0.9951) which means that the two features are almost same and therefore can be dropped. The inclusion of both in analysis or modeling might not create much value.

The other close correlation also lies between Total Backward Packet and Total Length of Bwd Packets (0.9615) and this is not surprising because, as a rule, more packets imply more total length. On the same note, Total Fwd Packets and Total Backward Packets (0.9498) are highly related indicating balanced communication flows.

Also, Total Fwd Packets is strongly correlated with Total Length of Bwd Packets (0.9200) and Packet Length Mean with Packet Length Std (0.8878), which means that there are no drastic variations in packet sizes. All these strong correlations are good in determining features to be used in the model to minimize redundancy and enhance efficiency of the model.

```
corr_pairs = correlation_matrix.unstack() # Convert the corr table to list
corr_pairs = corr_pairs[corr_pairs != 1] # Remove where corr is perfect 1
corr_pairs = corr_pairs.drop_duplicates() # Drop repeated pairs
corr_pairs_sorted = corr_pairs.abs().sort_values(ascending=False)
```

```
print("=" * 50)
print("  ↑ TOP 5 CORRELATED FEATURES")
print("=" * 50)
print(corr_pairs_sorted.head(5).to_string())
```

```
=====
  ↑ TOP 5 CORRELATED FEATURES
=====
Packet Length Mean      Average Packet Size      0.995068
Total Backward Packets  Total Length of Bwd Packets  0.961480
Total Fwd Packets       Total Backward Packets      0.949848
                        Total Length of Bwd Packets  0.920007
Packet Length Mean      Packet Length Std          0.887792
```

Figure no. 18 - top 5 correlated features

7. Section - 4 (Data Exploration)

An essential first step in learning how to examine a dataset is data exploration, which analyzes the format, quality, and inherent trends in a dataset (then executing any analytical or machine learning algorithm). In this instance, the dataset has network traffic data where a target variable is marked as normal traffic and attack. The main goals of data exploration are to understand how variables are distributed, to determine inconsistencies and to determine significant relationships among the features.

The dependent variable is the Label column and the distribution of the label column can help to determine the balance of the dataset. A balanced dataset will have approximately the same number of examples of each class, whereas an imbalanced dataset can distort predictive models. Preliminary analysis of the data shows that the Attack instances are by far more than the Normal Traffic and there is a sign of class imbalance. Such imbalance may influence performance in classification particularly in the normal behavior detection.

The next step would be to compute the summary statistics including mean, median, and standard deviation of the important numerical variables including Flow Duration, Packet Length Mean, and Fwd Packet Length Mean. These measures give us information on the central tendency of the data and the variability of the data. As an illustration, the larger values of Flow Duration in Attack class could indicate long suspicious activity whereas the fluctuations in packet length can indicate the different manners of traffic.

The data can be grouped by the Label column to compare the distributions of the features in the categories. This assists in determining the features that have a bigger impact on the distinction of normal and malicious traffic. Also, it is worth noting that one should look at outliers, which may distort analysis and influence the model. Attack traffic is usually more variable and contains more outliers than regular traffic.

All in all, data exploration demonstrates that the dataset is imbalanced and statistically significant differences between the classes of Normal Traffic and the Attack can be observed. These insights are the basis of additional visualization, testing of hypotheses and developing a model.

7a - Label Frequency Analysis

The frequency of the Label variable has two groups: the normal Traffic of 288,548 and the Attack of 199,531. This shows that there is more Normal Traffic than Attack in the dataset. Both classes are well represented, however not equally distributed. A difference of almost 89,000 records implies an average class balance. The best balanced data set would consist of roughly equal numbers of each category. But in this instance, Normal Traffic prevails, which can affect the classification model performance. The models that have been trained with this data can develop a bias towards the major category, which can decrease the level of Attack cases they can detect. Although this imbalance is not really very serious, it is still tangible enough to be addressed. It might require resampling, class-weighting, or application of other appropriate evaluation measures in order to provide sound model performance.

```
import matplotlib.pyplot as plt
```

```
label_counts = df_filtered['Label'].value_counts()
```

```
print("=" * 50)
print(" 📊 LABEL VALUE COUNTS")
print("=" * 50)
print(label_counts)
```

```
=====
📊 LABEL VALUE COUNTS
=====
Label
Normal Traffic    288548
Attack            199531
Name: count, dtype: int64
```

```
print("=" * 50)
print(" 📊 LABEL PERCENTAGES")
print("=" * 50)
print(round(label_counts / label_counts.sum() * 100, 2))
```

```
=====
📊 LABEL PERCENTAGES
=====
Label
Normal Traffic    59.12
Attack            40.88
Name: count, dtype: float64
```

Figure no. 19 - value counts

```
plt.figure(figsize=(6, 4))
plt.bar(label_counts.index, label_counts.values, color=['#c9b8f5', '#f5b8d0'], width=0.5)
plt.title('Frequency of Each Category in Label Column')
plt.xlabel('Label')
plt.ylabel('Frequency')
plt.show()
```

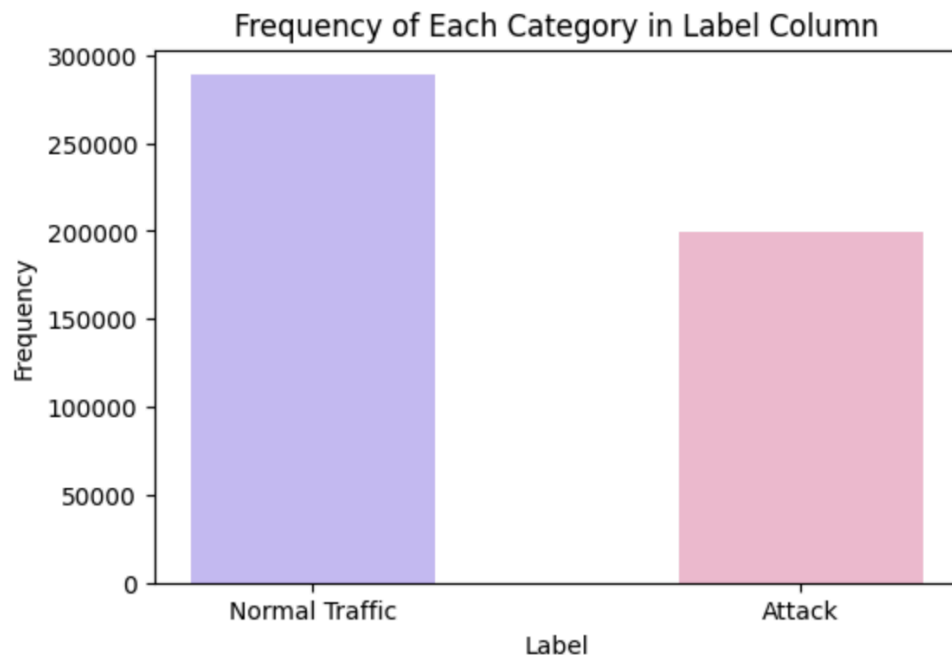


Figure no. 20 - bar chart of value counts

7b - Comparison of Average Flow Duration and Packet Length Mean

The computed means of the Flow Duration and Packet Length Mean are very minor between the Normal Traffic and Attack. In the case of Flow Duration, the average of the Normal Traffic (8.97×10^6) is a little higher than in the Attack (8.91×10^6). In a similar manner, in the case of Packet Length Mean, Attack possesses a slightly larger value (85.57) than in the case of Normal Traffic (85.54). These differences are slight which means that both of the categories have a very similar behavior in terms of these features.

Matplotlib pie charts are visually representative of the averages, however, since the data are close to each other, both categories of data take up almost the same percentage of the pie charts. It is not easy to tell the Normal Traffic and Attack based only on these two features. On the whole, the analysis indicates that Flow Duration and Packet Length Mean could not be powerful discriminative variables, and other variables might be required in order to achieve successful classification.

```
avg_flow = df_filtered.groupby('Label')['Flow Duration'].mean()
avg_packet = df_filtered.groupby('Label')['Packet Length Mean'].mean()
```

```
print("=" * 50)
print(" 🕒 AVERAGE FLOW DURATION")
print("=" * 50)
print(avg_flow)

print("\n")

print("=" * 50)
print(" 📦 AVERAGE PACKET LENGTH MEAN")
print("=" * 50)
print(avg_packet)
```

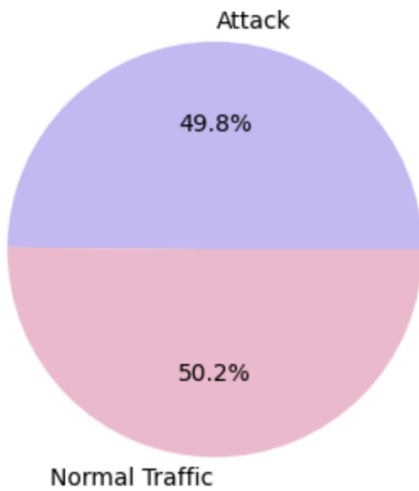
```
=====
🕒 AVERAGE FLOW DURATION
=====
Label
Attack          8.909046e+06
Normal Traffic   8.966348e+06
Name: Flow Duration, dtype: float64

=====
📦 AVERAGE PACKET LENGTH MEAN
=====
Label
Attack          85.573255
Normal Traffic   85.542480
Name: Packet Length Mean, dtype: float64
```

Figure no. 21 - Average flow and packet mean

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.pie(avg_flow.values, labels=avg_flow.index,
        colors=['#c9b8f5', '#f5b8d0'], autopct='%1.1f%%')
ax1.set_title('Average Flow Duration by Label')
ax2.pie(avg_packet.values, labels=avg_packet.index,
        colors=['#c9b8f5', '#f5b8d0'], autopct='%1.1f%%')
ax2.set_title('Average Packet Length Mean by Label')
plt.show()
```

Average Flow Duration by Label



Average Packet Length Mean by Label

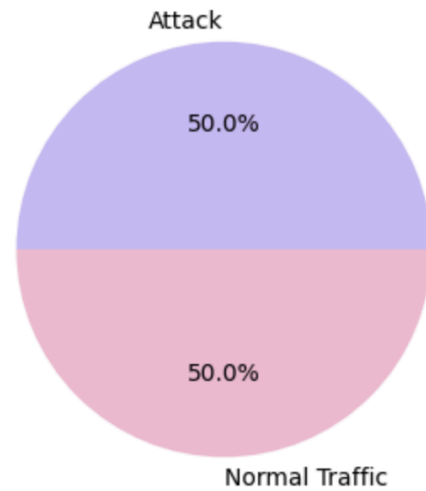


Figure no. 22 - bar chart

7c - Boxplot of 'Fwd Packet Length Mean' grouped by the Label column

To analyze the distribution of values of both the Normal Traffic and Attack, a boxplot of Fwd Packet Length Mean by the Label column was created. The visualization itself indicates the important statistical characteristics of the median, interquartile range (IQR), and the existence of the outliers of each category. Based on the boxplot, it can be seen that both the figures of Normal Traffic and Attack are similar in terms of the median, which means that their central tendencies are similar. The Attack category however displays a broader distribution where the interquartile range is larger and the extreme outliers are larger as well. This is an indication that attack traffic has more variability in forward packet length, which may be attributed to different behaviors and patterns of attacks. Conversely, the Normal Traffic is more regular, has a smaller dispersion, and fewer exceptions. This is more predictable and stable network behavior.

```
normal = df_filtered[df_filtered['Label'] == 'Normal Traffic']['Fwd Packet Length Mean']
attack = df_filtered[df_filtered['Label'] == 'Attack']['Fwd Packet Length Mean']
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
plt.figure(figsize=(6, 3))
plt.boxplot([normal, attack], labels=['Normal Traffic', 'Attack'])
plt.title('Fwd Packet Length Mean by Label')
plt.xlabel('Label')
plt.ylabel('Fwd Packet Length Mean')
plt.show()
```

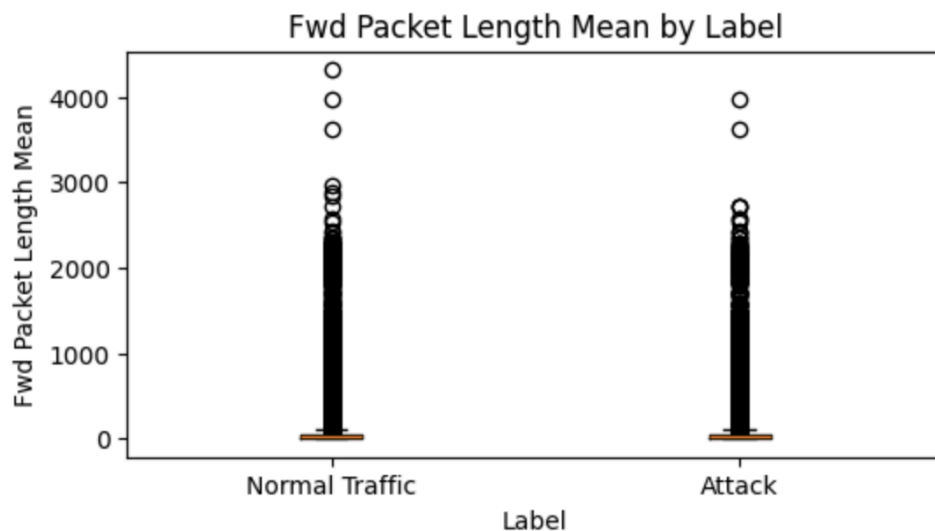


Figure no. 23 - boxplot of fwd packet length

7d - Hypothesis test to determine difference in the mean Flow Duration

An independent t-test was used to ascertain whether or not there is a significant difference between the mean Flow Duration of Normal Traffic and Attack classes. The null hypothesis (H0) was that the mean Flow Duration was the same whereas the alternative one was that there was a significant difference between the two. The result of the test was t-statistic 0.716 and p-value was 0.474. The p-value exceeds the significance of 0.05 and therefore we cannot reject the null hypothesis. This implies that the difference in Flow Duration of the two categories is not statistically significant and hence both the normal and the attack traffic have very similar average flow durations. Therefore, Flow Duration might not be a powerful distinguishing factor and more network features ought to be incorporated in order to be successful in classification and intrusion detection.

```
from scipy import stats
```

```
normal = df_filtered[df_filtered['Label'] == 'Normal Traffic']['Flow Duration']  
attack = df_filtered[df_filtered['Label'] == 'Attack']['Flow Duration']
```

```
t_stat, p_value = stats.ttest_ind(normal, attack)
```

```
print("=" * 50)  
print(" 📊 T-STATISTIC")  
print("=" * 50)  
print(t_stat)
```

```
print("\n")
```

```
print("=" * 50)  
print(" 📉 P-VALUE")  
print("=" * 50)  
print(p_value)
```

```
=====
📊 T-STATISTIC
=====
0.7161327588416806
```

```
=====
📉 P-VALUE
=====
0.473909720385436
```

Figure no. 24 - hypothesis testing

```

print("=" * 50)
print("  🧪 HYPOTHESIS TEST RESULT")
print("=" * 50)
if p_value < 0.05:
    print("  ✅ Result: Reject Null Hypothesis")
    print("  There is a significant difference")
else:
    print("  ❌ Result: Accept Null Hypothesis")
    print("  There is no significant difference")

```

```

=====
  🧪 HYPOTHESIS TEST RESULT
=====

```

```

❌ Result: Accept Null Hypothesis
There is no significant difference

```

Figure no. 25 - hypothesis test result

7e - Dashboard

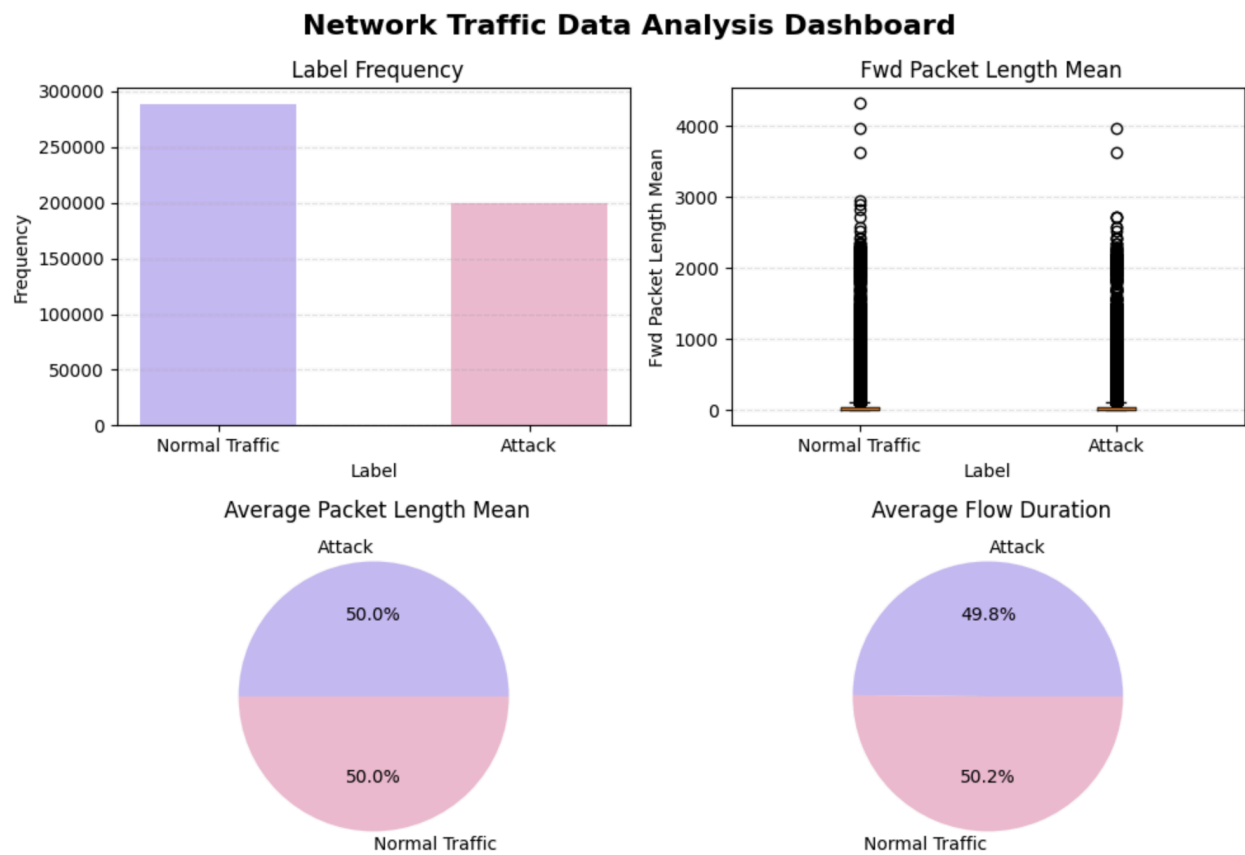


Figure no. 26 - Dashboard

8. Summary

The main aim of the study was to comprehend, prepare, examine and dig into the data to establish trends that would distinguish between a Normal Traffic and an Attack traffic. The data set utilized in this analysis had 288, 602 records and 79 features, which are different features of network flows including packet size, flow duration and number of packets. Out of the initial data set 16 applicable features were selectively taken to make sure that the samples of flow-based, packet-based and time-based characteristics were balanced. This move made the dataset easier to use and meaningful information was still left to be used later.

During the data preparation phase, a number of preprocessing procedures were done to enhance the quality of data. The missing values were detected with the help of the appropriate methods, and 36 missing records were eliminated in order to make the data set clean and credible. Further, it was noticed that duplicate records were found showing a high number of duplicate records. These issues had to be addressed in order to exclude biased or misleading results. The Label column was also converted into more readable variables- "Normal Traffic" and "Attack" which simplify the process of understanding and analysis of the data.

The data analysis stage was based on correlation analysis and statistical analysis. The key characteristics like Flow Duration and Packet Length Mean were determined and summary statistics were determined including mean, standard deviation, skewness, and the kurtosis. The findings revealed that the data was strongly skewed and there were outliers, which implied the variability of the network behavior. Correlation showed the existence of strong relationships between some of the features, including Total Forward Packets and Total Backward Packets, Packet Length Mean and Average Packet Size. These results indicated possible redundancy in the data and gave information on how features can be selected in the future when the model is being developed.

A more detailed interpretation of the data was performed in the data exploration stage by drawing bar charts, pie charts, and boxplots to analyze the data better. The class distribution analysis indicated the existence of a moderate imbalance in the classes, with the Attack traffic slightly overpowering the Normal Traffic. A comparison between Flow Duration and Packet Length Mean showed that there is not much difference in the two classes, meaning that both might not be a good characteristic in classification. Nonetheless, the boxplot of Forward Packet Length Mean showed a wider variation in Attack traffic implying that there might be some characteristics that can be used to obtain valuable distinguishing patterns.

Lastly, a hypothesis was tested by independent t-test to determine the existence of statistically significant differences between the Flow Duration of Normal Traffic and Attack. The resultant p-value (0.47) showed that there was no significant difference, which strengthened the argument that Flow Duration is not a very strong discriminating characteristic.

9. Conclusion

The current course work has provided a detailed understanding of the whole data analysis process, data collection and data preparation, data analysis and interpretation. Through the Smart Data Discovery module, I got a chance to deal with real-world data, which gave me the necessary experience in technical and analytical expertise. The project entailed application of theoretical knowledge into practice and I was in a position to comprehend the concept of data handling, pre processing and exploratory analysis further.

One of the most helpful and beneficial things in this course work was cleaning and preparation of raw data. The identification of the absent values, deletion of the duplicates and the choice of the features of interest presented the importance of the quality of data used in drawing credible conclusions. The statistical analysis and correlation analysis process also enabled me to understand the interaction between different variables, and how useful information could be quantified with the help of complicated data.

The essential role in the achievement of this project was played by Python and Python libraries such as Pandas, Matplotlib, Seaborn, and SciPy. It advanced my performance of programming and improved my skills on using data analysis tools in an orderly and effective manner. My skills of data interpretation and presentation of findings were also improved through the development of visualizations and hypothesis testing.

The other important soft skills that this coursework helped me to develop include problem-solving, critical thinking and time management. One had to think through each step of the project, to be attentive to details and troubleshoot errors.